Copy

Debugging

Many tools are at your disposal for debugging Pintos. This section introduces a few of them to you.

printf()

Don't underestimate the value of `printf()`.

- The way `printf()` is implemented in Pintos, **you can call it from practically anywhere in the kernel, whether it's in a kernel thread or an interrupt handler**, almost regardless of what locks are held.

`printf()` is useful for more than just examining data. It can also help figure out when and where something goes wrong, even when the kernel crashes or panics without a useful error message.

- The strategy is to sprinkle calls to `printf()` with different strings (e.g. `"<1>"`, `"<2>"`, ...) throughout the pieces of code you suspect are failing. If you don't even see `<1>` printed, then something bad happened before that point, if you see `<1>` but not `<2>`, then something bad happened between those two points, and so on.
- Based on what you learn, you can then insert more `printf()` calls in the new, smaller region of code you suspect. Eventually, you can **narrow the problem down** to a single statement. See the following expansion ***Triple Faults***, for a related technique.

> Triple Faults

ASSERT

Assertions are useful because they can catch problems early, before they'd otherwise be noticed.

- **Ideally, each function should begin with a set of assertions that check its arguments for validity.** (Initializers for functions' local variables are evaluated before assertions are checked, so be careful not to assume that an argument is valid in an initializer.)
- You can also sprinkle assertions throughout the body of functions in places where you suspect things are likely to go wrong. They are especially useful for checking loop invariants.

Pintos provides the `ASSERT` macro, defined in `<debug.h>`, for checking assertions.

- **Macro: ASSERT (*expression*)**
  - Tests the value of expression. If it evaluates to zero (false), the kernel panics. The panic message includes the expression that failed, its file and line number, and a backtrace, which should help you to find the problem. See the following expansion ***Backtraces***, for more information.

> Backtraces

> Backtraces Example

Function and Parameter Attributes

These macros defined in `<debug.h>` tell the compiler special attributes of a function or function parameter. Their expansions are GCC-specific.

- **Macro: UNUSED**
  - Appended to a function parameter to tell the compiler that the parameter might not be used within the function. It suppresses the warning that would otherwise appear.
- **Macro: NO_RETURN**
  - Appended to a function prototype to tell the compiler that the function never returns. It allows the compiler to fine-tune its warnings and its code generation.
- **Macro: NO_INLINE**
  - Appended to a function prototype to tell the compiler to never emit the function in-line. Occasionally useful to improve the quality of backtraces (see below).
- **Macro: PRINTF_FORMAT** _(format, first)**_
  - Appended to a function prototype to tell the compiler that the function takes a `printf()`-like format string as the argument numbered *format* (starting from 1) and that the corresponding value arguments start at the argument numbered *first*. This lets the compiler tell you if you pass the wrong argument types.

GDB

You can run Pintos under the supervision of the GDB debugger.

- First, start Pintos with the `--gdb` option, e.g. `pintos --gdb -- run mytest`.

You should find your Pintos program blocked, which waits for the GDB to attach to this gdb session.

- Second, **open a second terminal on the same machine (or in the same container, see below)** and use `pintos-gdb` to invoke GDB on `kernel.o`:

> If you develop Pintos in **a virtual machine**, it is easy to do the second step above by opening another terminal.
>
> If you use **docker**, you need a way to attach your Pintos container to another terminal in your host computer.
>
> - Now **open a new terminal and run**
>
> ```
> docker exec -it pintos bash
> ```
>
> You may remember that when you launch the Pintos docker container , you name it as "pintos" by specifying "--name pintos". So here you just exec a bash shell in your pintos container.

- Third, use `pintos-gdb` to invoke GDB on `kernel.o` in the second terminal:

```
cd pintos/src/threads/build
pintos-gdb kernel.o
```

and issue the following GDB command:

```
debugpintos
```

A GDB macro `debugpintos` is provided as a shorthand for `target remote localhost:1234`, so you can type this shorter command (`debugpintos`) instead.

- Now GDB is connected to the simulator over a local network connection. **You can now issue any normal GDB commands.**
  - If you issue the "c" (continue) command, the simulated Qemu will take control, load Pintos, and then Pintos will run in the usual way. You can pause the process at any point with `Ctrl+C`.

Using GDB

You can read the GDB manual by typing `info gdb` at a terminal command prompt. Here's a few commonly useful GDB commands:

- **GDB Command: c**
  - Continues execution until Ctrl+C or the next breakpoint.
- **GDB Command: si**
  - Execute one machine instruction.
- **GDB Command: s**
  - Execute until next line reached, step into function calls.
- **GDB Command: n**
  - Execute until next line reached, step over function calls.
- **GDB Command: p** ***expression***
  - Evaluates the given expression and prints its value. If the expression contains a function call, that function will actually be executed.
- **GDB Command: finish**
  - Run until the selected function (stack frame) returns
- **GDB Command: b** ***function***
- **GDB Command: b** ***file:line***
- **GDB Command: b *** ***address***
  - Sets a breakpoint at *function*, at *line* within *file*, or *address*. `b` is short for `break` or `breakpoint`. (Use a 0x prefix to specify an address in hex.)
  - Use `b pintos_init` to make GDB stop when Pintos starts running.
- **GDB Command: info** ***registers***
  - Print the general purpose registers, eip, eflags, and the segment selectors. For a much more thorough dump of the machine register state, see QEMU's own info registers command.
- **GDB Command: x/Nx** ***addr***

    - Display a hex dump of N words starting at virtual address *addr*. If N is omitted, it defaults to 1. *addr* can be any expression.
- **GDB Command: x/Ni *addr***
    - Display the N assembly instructions starting at *addr*. Using $eip as *addr* will display the instructions at the current instruction pointer.
- **GDB Command: l **address***
    - Lists a few lines of code around *address*. (Use a 0x prefix to specify an address in hex.)
- **GDB Command: bt**
    - Prints a stack backtrace similar to that output by the `backtrace` program described above.
- **GDB Command: frame *n***
    - Select frame number n or frame at address n
- **GDB Command: p/a *address***
    - Prints the name of the function or variable that occupies *address*. (Use a 0x prefix to specify an address in hex.)
- **GDB Command: diassemble *function***
    - Disassembles function.
- **GDB Command: thread *n***
    - GDB focuses on one thread (i.e., CPU) at a time. This command switches that focus to thread n, numbered from zero.
- **GDB Command: info *threads***
    - List all threads (i.e., CPUs), including their state (active or halted) and what function they're in.

We also provide a set of macros specialized for debugging Pintos, written by Godmar Back gback@cs.vt.edu. You can type `help user-defined` for basic help with the macros. Here is an overview of their functionality, based on Godmar's documentation:

- **GDB Macro: debugpintos**

- Attach debugger to a waiting pintos process on the same machine. Shorthand for `target remote localhost:1234`.

- **GDB Macro: dumplist &*list type element***
  - Prints the elements of *list*, which should be a `struct` *list* that contains elements of the given *type* (without the word `struct`) in which *element* is the `struct list_elem` member that links the elements.
  - Example: `dumplist &all_list thread allelem` prints all elements of `struct thread` that are linked in `struct list all_list` using the `struct list_elem allelem` which is part of `struct thread`.
- **GDB Macro: btthread *thread***
  - Shows the backtrace of *thread*, which is a pointer to the `struct thread` of the thread whose backtrace it should show. For the current thread, this is identical to the `bt` (backtrace) command. It also works for any thread suspended in `schedule()`, provided you know where its kernel stack page is located.
- **GDB Macro: btthreadlist *list element***
  - Shows the backtraces of all threads in *list*, the `struct list` in which the threads are kept. Specify element as the `struct list_elem` field used inside `struct thread` to link the threads together.
  - Example: `btthreadlist all_list allelem` shows the backtraces of all threads contained in `struct list all_list`, linked together by `allelem`. This command is useful to determine where your threads are stuck when a deadlock occurs. Please see the example scenario below.
- **GDB Macro: btpagefault**
  - Print a backtrace of the current thread after a page fault exception. Normally, when a page fault exception occurs, GDB will stop with a message that might say:

```
Program received signal 0, Signal 0.
0xc0102320 in intr0e_stub ()
```

In that case, the `bt` command might not give a useful backtrace. Use `btpagefault` instead.

You may also use `btpagefault` for page faults that occur in a user process. In this case, you may wish to also load the user program's symbol table using the `loadusersymbols` macro, as described below.

- **GDB Macro: loadusersymbols**
  - You can also use GDB to debug a user program running under Pintos. To do that, use the `loadusersymbols` macro to load the program's symbol table:

```
loadusersymbols program
```

where program is the name of the program's executable (in the host file system, not in the Pintos file system). For example, you may issue:

```
(gdb) loadusersymbols tests/userprog/exec-multiple
add symbol table from file "tests/userprog/exec-
multiple" at
    .text_addr = 0x80480a0
(gdb)
```

After this, you should be able to debug the user program the same way you would the kernel, by placing breakpoints, inspecting data, etc. Your actions apply to every user program running in Pintos, not just to the one you want to debug, so be careful in interpreting the results: GDB does not know which process is currently active (because that is an abstraction the Pintos kernel creates). Also, a name that appears in both the kernel and the user program will actually refer to the kernel name. (The latter problem can be avoided by giving the user executable name on the GDB command line, instead of kernel.o, and then using `loadusersymbols` to load kernel.o.) `loadusersymbols` is implemented via GDB's `add-symbol-file` command.

- **GDB Macro: hook-stop**

- GDB invokes this macro every time the simulation stops, which Bochs will do for every processor exception, among other reasons. If the simulation stops due to a page fault, `hook-stop` will print a message that says and explains further whether the page fault occurred in the kernel or in user code.
- If the exception occurred from user code, `hook-stop` will say:

```
pintos-debug: a page fault exception occurred in user
mode
pintos-debug: hit 'c' to continue, or 's' to step to
intr_handler
```

In Project 2, a page fault in a user process leads to the termination of the process. You should expect those page faults to occur in the robustness tests where we test that your kernel properly terminates processes that try to access invalid addresses. To debug those, set a break point in `page_fault()` in exception.c, which you will need to modify accordingly.

In Project 3, a page fault in a user process no longer automatically leads to the termination of a process. Instead, it may require reading in data for the page the process was trying to access, either because it was swapped out or because this is the first time it's accessed. In either case, you will reach `page_fault()` and need to take the appropriate action there.

If the page fault did not occur in user mode while executing a user process, then it occurred in kernel mode while executing kernel code. In this case, `hook-stop` will print this message:

```
pintos-debug: a page fault occurred in kernel mode
```

followed by the output of the `btpagefault` command.

> Before Project 2, a page fault exception in kernel code is always a bug in your kernel, because your kernel should never crash. Starting with Project 2, the situation will change if you use the `get_user()` and `put_user()` strategy to verify user memory accesses (If you are don't know what does this mean, don't worry, you should understand when you work on Project 2.)

Example GDB Session

This section narrates a sample GDB session, provided by Godmar Back. This example illustrates how one might debug a Project 1 solution in which occasionally a thread that calls `timer_sleep()` is not woken up. With this bug, tests such as `mlfqs_load_1` get stuck.

This session was captured with a slightly older version of Bochs and the GDB macros for Pintos, so it looks slightly different than it would now. Program output is shown in normal type, user input is after the "$" or "(gdb)".

- First, I **start Pintos**:

```
$ pintos -v --gdb -- -q -mlfqs run mlfqs-load-1
Writing command line to /tmp/gDAlqTB5Uf.dsk...
bochs -q
==============================================================
                     Bochs x86 Emulator 2.2.5
             Build from CVS snapshot on December 30, 2005
==============================================================
00000000000i[     ] reading configuration from bochsrc.txt
00000000000i[     ] Enabled gdbstub
00000000000i[     ] installing nogui module as the Bochs GUI
00000000000i[     ] using log file bochsout.txt
Waiting for gdb connection on localhost:1234
```

- Then, I **open a second window on the same machine (or container) and start GDB**:

```
$ pintos-gdb kernel.o
GNU gdb Red Hat Linux (6.3.0.0-1.84rh)
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public
License, and you are
welcome to change it and/or distribute copies of it
under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB.  Type "show
warranty" for details.
This GDB was configured as "i386-redhat-linux-gnu"...
Using host libthread_db library
"/lib/libthread_db.so.1".
```

- Then, I **tell GDB to attach to the waiting Pintos emulator**:

```
(gdb) debugpintos
Remote debugging using localhost:1234
0x0000fff0 in ?? ()
Reply contains invalid hex digit 78
```

- Now I tell Pintos to run by executing `c` (short for `continue`) twice:

```
(gdb) c
Continuing.
Reply contains invalid hex digit 78
(gdb) c
Continuing.
```

- Now Pintos will continue and output:

```
Pintos booting with 4,096 kB RAM...
Kernel command line: -q -mlfqs run mlfqs-load-1
374 pages available in kernel pool.
373 pages available in user pool.
Calibrating timer...  102,400 loops/s.
Boot complete.
Executing 'mlfqs-load-1':
(mlfqs-load-1) begin
(mlfqs-load-1) spinning for up to 45 seconds, please
wait...
(mlfqs-load-1) load average rose to 0.5 after 42 seconds
(mlfqs-load-1) sleeping for another 10 seconds, please
wait...
```

- ...until it gets stuck because of the bug I had introduced. I hit `Ctrl+C` in the debugger window:

```
Program received signal 0, Signal 0.
0xc010168c in next_thread_to_run () at
../../threads/thread.c:649
649   while (i <= PRI_MAX && list_empty
(&ready_list[i]))
(gdb)
```

- The thread that was running when I interrupted Pintos was the idle thread. If I run `backtrace`, it shows this backtrace:

```
(gdb) bt
#0  0xc010168c in next_thread_to_run () at
../../threads/thread.c:649
#1  0xc0101778 in schedule () at
../../threads/thread.c:714
#2  0xc0100f8f in thread_block () at
../../threads/thread.c:324
#3  0xc0101419 in idle (aux=0x0) at
../../threads/thread.c:551
#4  0xc010145a in kernel_thread (function=0xc01013ff ,
aux=0x0)
    at ../../threads/thread.c:575
#5  0x00000000 in ?? ()
```

Not terribly useful. What I really like to know is what's up with the other thread (or threads). Since I keep all threads in a linked list called `all_list`, linked together by a `struct list_elem` member named `allelem`, I can use the `btthreadlist` macro from the macro library I wrote. `btthreadlist` iterates through the list of threads and prints the backtrace for each thread:

```
(gdb) btthreadlist all_list allelem
pintos-debug: dumping backtrace of thread 'main'
@0xc002f000
#0  0xc0101820 in schedule () at
../../threads/thread.c:722
#1  0xc0100f8f in thread_block () at
../../threads/thread.c:324
#2  0xc0104755 in timer_sleep (ticks=1000) at
../../devices/timer.c:141
#3  0xc010bf7c in test_mlfqs_load_1 () at
../../tests/threads/mlfqs-load-1.c:49
#4  0xc010aabb in run_test (name=0xc0007d8c "mlfqs-load-
1")
    at ../../tests/threads/tests.c:50
#5  0xc0100647 in run_task (argv=0xc0110d28) at
../../threads/init.c:281
#6  0xc0100721 in run_actions (argv=0xc0110d28) at
../../threads/init.c:331
#7  0xc01000c7 in main () at ../../threads/init.c:140

pintos-debug: dumping backtrace of thread 'idle'
@0xc0116000
#0  0xc010168c in next_thread_to_run () at
../../threads/thread.c:649
#1  0xc0101778 in schedule () at
../../threads/thread.c:714
#2  0xc0100f8f in thread_block () at
../../threads/thread.c:324
#3  0xc0101419 in idle (aux=0x0) at
../../threads/thread.c:551
#4  0xc010145a in kernel_thread (function=0xc01013ff ,
aux=0x0)
    at ../../threads/thread.c:575
#5  0x00000000 in ?? ()
```

In this case, there are only two threads, the idle thread and the main thread. The kernel stack pages (to which the `struct thread` points) are at 0xc0116000 and 0xc002f000, respectively. The main thread is stuck in `timer_sleep()`, called from `test_mlfqs_load_1`.

Knowing where threads are stuck can be tremendously useful, for instance when diagnosing deadlocks or unexplained hangs.

FAQ

GDB can't connect to Bochs.

If the `target remote` command fails, then make sure that both GDB and `pintos` are running on the same machine (or container) by running `hostname` in each terminal. If the names printed differ, then you need to open a new terminal for GDB on the machine running `pintos`.

GDB doesn't recognize any of the macros.

If you start GDB with `pintos-gdb`, it should load the Pintos macros automatically. If you start GDB some other way, then you must issue the command `source pintosdir/src/misc/gdb-macros`, where pintosdir is the root of your Pintos directory, before you can use them.

Can I debug Pintos with DDD?

Yes, you can. DDD invokes GDB as a subprocess, so you'll need to tell it to invokes `pintos-gdb` instead:

```
ddd --gdb --debugger pintos-gdb
```

Can I use GDB inside Emacs?

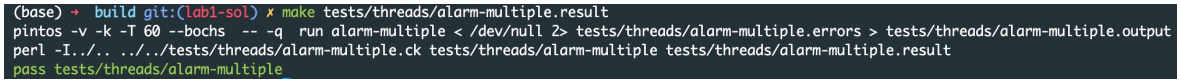
Yes, you can. Emacs has special support for running GDB as a subprocess. Type `M-x gdb` and enter your `pintos-gdb` command at the prompt. The Emacs manual has information on how to use its debugging features in a section titled "Debuggers."

GDB is doing something weird.

> If you notice strange behavior while using GDB, there are three possibilities: a bug in your modified Pintos, a bug in Bochs's interface to GDB or in GDB itself, or a bug in the original Pintos code. The first and second are quite likely, and you should seriously consider both. We hope that the third is less likely, but it is also possible.

Using GDB to Debug a Single Test

If you issue the `make` command to run a single test (see Run an individual test), you will see the command our test suite uses to generate `.result` output (see the example below). With this command added `--gdb` option before `--`, you can run this single test in gdb mode and then debug with `pintos-gdb` as above.



Tips

The page allocator in `threads/palloc.c` and the block allocator in `threads/malloc.c` clear all the bytes in memory to 0xcc at time of free.

- Thus, if you see an attempt to dereference a pointer like 0xcccccccc, or some other reference to 0xcc, there's a good chance you're trying to reuse a page that's already been freed.
- Also, byte 0xcc is the CPU opcode for "invoke interrupt 3," so if you see an error like `Interrupt 0x03 (#BP Breakpoint Exception)`, then Pintos tried to execute code in a freed page or block.

An assertion failure on the expression `sec_no < d->capacity` indicates that:

- Pintos tried to access a file through an inode that has been closed and freed. Freeing an inode clears its starting sector number to 0xcccccccc, which is not a valid sector number for disks smaller than about 1.6 TB.

Previous
Testing

Next
Grading

Last updated 1 year ago